

DATA VORTEX 2026

Round 1 — Phase 2

Logic Explanation

Team Name – akarshh2456

Selected Questions: E3 (Easy), M3 (Medium), H4 (Hard)

E3 — Average Engagement by Platform

Objective: Determine how effectively each identified social-media platform generates engagement on an average post. The analysis calculates average likes, shares, comments, and overall engagement for every platform and ranks platforms by average total engagement.

Step-by-Step Logic

1. Select the platform dimension: The platform column is the grouping dimension because the question asks for a platform-level comparison.
2. Handle missing platform values: The condition `platform IS NOT NULL AND platform <> 'Unknown'` removes missing/unknown platform records. In Phase 1, missing platforms were represented as 'Unknown', so including them as a real platform would distort the comparison.
3. Calculate component averages: `AVG(likes)`, `AVG(shares)`, and `AVG(comments)` show the typical contribution of each engagement type on a platform.
4. Calculate total engagement: For every post, total engagement is defined as likes + shares + comments. `AVG(likes + shares + comments)` therefore gives the average engagement per post for each platform.
5. Aggregate by platform: `GROUP BY platform` produces one result row for each identified platform.
6. Rank platforms: `ORDER BY avg_total_engagement DESC` places the platform with the highest average engagement first.
7. Why this approach: Raw post counts can favour platforms simply because they have more records. Average engagement provides a normalized comparison of how much engagement a typical post receives.

M3 — Most Active Users

Objective: Identify the ten users who contribute the largest number of posts, while also showing their audience size, location, and cumulative engagement.

Step-by-Step Logic

1. Combine the two tables: The users table contains user attributes such as `follower_count` and `location`, while posts contains individual post activity. They are connected through `user_id`.
2. Use an INNER JOIN: `JOIN users u ON u.user_id = p.user_id` matches each post to its corresponding user. This allows post-level activity to be analysed together with user-level attributes.
3. Count posts: `COUNT(p.post_id)` calculates how many posts each user has made. This is the primary definition of 'most active' in the question.
4. Calculate total engagement: `SUM(likes + shares + comments)` adds engagement across all posts belonging to each user.

5. Group at user level: GROUP BY user_id, follower_count, and location creates one aggregated record per user.
6. Rank the users: ORDER BY post_count DESC sorts users from the highest number of posts to the lowest. total_engagement DESC is a secondary ordering rule to make ties deterministic and favour the more impactful user.
7. Return only the top ten: LIMIT 10 restricts the final output to the ten most active users.
Why this approach: The query distinguishes activity from popularity. A user can have many followers without being an active poster, so post count is used as the primary activity metric while engagement provides additional context.

H4 — Follower-to-Engagement Anomaly

Objective: Find users with fewer than 5,000 followers whose total engagement nevertheless places them in the highest 10% of all users by engagement. These users can be viewed as high-impact users relative to their relatively small audiences.

Step-by-Step Logic

1. Build user-level engagement: The first CTE, user_totals, joins users and posts and aggregates all posts belonging to each user. SUM(likes + shares + comments) produces each user's total engagement.
2. Preserve relevant user attributes: user_id, location, and follower_count are retained so the final anomaly records can be interpreted and investigated.
3. Create engagement deciles: The second CTE uses NTILE(10) OVER (ORDER BY total_engagement DESC). NTILE(10) divides the ordered user population into ten approximately equal groups.
4. Interpret decile 1: Because users are ordered from highest to lowest engagement, engagement_decile = 1 represents the highest-engagement 10% segment.
5. Apply the audience-size condition: follower_count < 5000 identifies users with comparatively small audiences.
6. Combine both conditions: The WHERE clause requires both a small follower count and membership in the top engagement decile. This is what makes the result an engagement anomaly rather than merely a list of popular users.
7. Rank the detected anomalies: ORDER BY total_engagement DESC places the strongest high-impact users first.
Why NTILE(10): The question asks for the top 10% rather than a fixed engagement threshold. NTILE(10) is therefore a direct SQL approach for segmenting the user population into ten ranked groups without hardcoding an engagement cutoff.
8. Analytical interpretation: A user meeting these conditions generates exceptionally high total engagement despite having fewer than 5,000 followers. Such accounts may be unusually effective at generating interaction and are useful candidates for further behavioural investigation.

Overall Analytical Approach

The three selected questions progressively demonstrate SQL analytical reasoning. E3 performs platform-level aggregation and normalization; M3 combines relational data and summarizes user behaviour; H4 applies a window function to identify a top-percentile engagement segment and then detects users whose engagement is disproportionately high relative to follower count. Together, the queries demonstrate filtering, aggregation, joins, ordering, limiting, common table expressions, and window functions.